Project Brief: Reading List (TanStack Query Mutations Only)

Overview

Build a small Reading List app where you add books, change their status, rate them and remove them. The app is deliberately plain, because the project is only about **mutations** in TanStack Query. It has five stages, and each stage practices one mutation pattern: adding with a refetch, updating, removing with an optimistic update and rollback, rating, and handling errors. Plan for 2 to 3 days. Use a made-up app name and made-up books.

Rules

- Vite + React, with TanStack Query v5. TypeScript preferred, JavaScript allowed. Tailwind and shadcn/ui are optional, because the focus is mutations, not styling.
- Every change to server data must use `useMutation`. No `fetch` calls inside click handlers.
- Every read of server data must use `useQuery`. No `useEffect` + `fetch`.
- Put all request functions in one `api` file. Each one must check `response.ok` and **throw** when it is false. `fetch` does not throw on a 404 or 500 by itself, so without this a failed mutation would look like a success and `onError` would never run.
- Build query keys with one small factory, for example `bookKeys.all` and `bookKeys.detail(id)`.
- No Redux, Zustand or other state libraries for server data.

Set up

Run these in a terminal. Node 20 or newer is recommended.

```
npm create vite@latest reading-list -- --template react-ts
cd reading-list
npm install
npm install @tanstack/react-query @tanstack/react-query-devtools
npm install -D json-server@0.17.4
```

Create `db.json` in the project root:

```
{ "books": [] }
```

Create `fail.js` in the project root. It makes the fake server reject any change request that carries the header `x-fail: true`, so he can test error handling without stopping the server:

```
module.exports = (req, res, next) => {
  if (req.method !== "GET" && req.headers["x-fail"] === "true") {
    return res.status(500).json({ error: "Forced failure" });
  }
  next();
};
```

Add this script to `package.json`. The 800 ms delay makes pending states and optimistic updates easy to see:

```
"scripts": {
  "api": "json-server --watch db.json --port 3001 --delay 800 --middlewares
./fail.js"
}
```

Run the app and the fake API in two terminals:

```
npm run dev
npm run api
```

Then wrap the app in `QueryClientProvider` in `main.tsx` and add `ReactQueryDevtools` so he can watch the cache change.

Endpoints

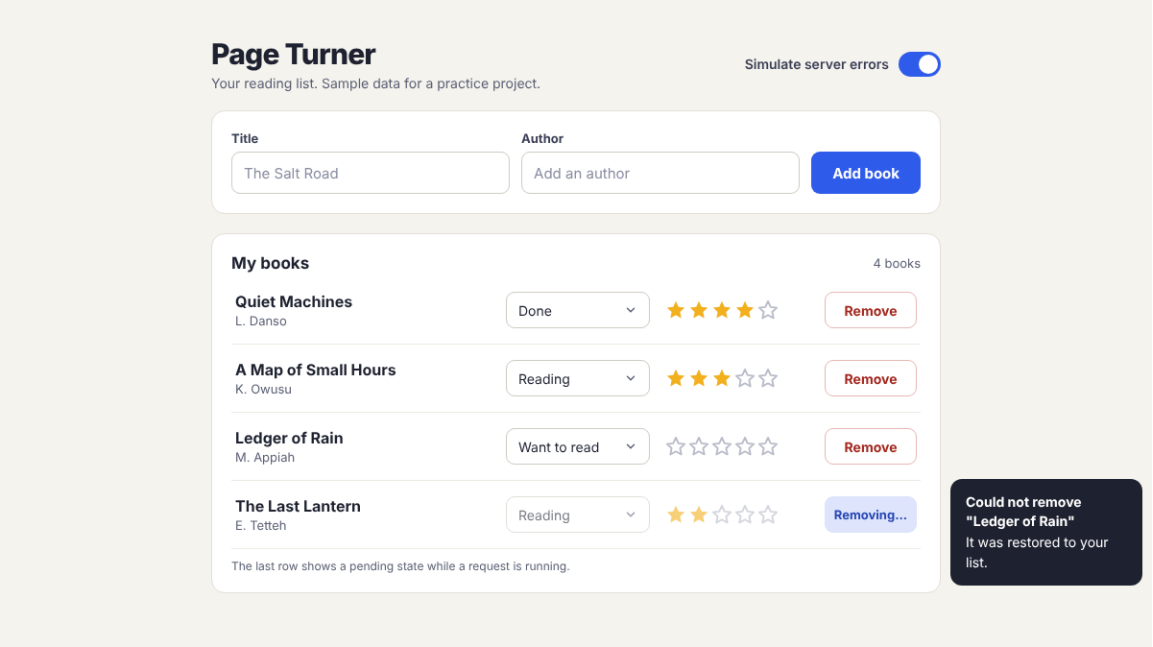
Base URL: `http://localhost:3001`. A book looks like `{ id, title, author, status, rating }`, where `status` is `want`, `reading` or `done` and `rating` is 0 to 5. json-server creates the `id` for new books.

Method and path	Body	Used for
`GET /books`	none	Show the list (a query)
`POST /books`	`{ title, author, status: "want", rating: 0 }`	Stage 1: add a book
`PATCH /books/:id`	`{ status }`	Stage 2: change status
`DELETE /books/:id`	none	Stage 3: remove a book
`PATCH /books/:id`	`{ rating }`	Stage 4: rate a book

To force an error, add the header `x-fail: true` to any change request (POST, PATCH or DELETE). `GET` requests are never rejected.

Target layout

This is how the finished app should look. It shows the add form, the error switch, each row's status, stars and Remove button, one row in a pending state, and an error toast after a failed removal. The app name, books and authors are made up. Styling does not have to match, but every part shown must exist and work.



Stages

Build the stages in order. Do not move on until the "Prove it" checks pass.

Stage 1: Add a book (refetch after success)

Build: a form with a title field, an author field and an Add button. Submitting runs a `useMutation` that posts the book. On success, invalidate the books query so the list reloads.

- While the mutation is pending, the button is disabled and says "Adding...".
- After success, the form clears.
- On error, show an error message under the form.

Prove it: add a book and it appears in the list without refreshing. Click Add twice very fast and only one book is saved. Remove the `invalidateQueries` line and explain what changes.

Stage 2: Change status (update with a refetch)

Build: a dropdown on each book to switch between Want to read, Reading and Done. Each change runs a mutation that patches `status`, then invalidates the list.

- Only the row being changed is disabled while pending. The other rows stay usable.

Prove it: change two different books quickly. Both end up with the right status. Explain why the key you invalidate must match the key the list uses.

Stage 3: Remove a book (optimistic update with rollback)

Build: a Remove button on each row. The book disappears from the screen immediately, before the server answers. Use these callbacks:

1. `onMutate`: cancel any running books query, save a copy of the current list, then remove the book from the cache.
2. `onError`: put the saved copy back.
3. `onSettled`: invalidate the books query.

Prove it: with the error switch off, the book vanishes instantly and stays gone. With the error switch on, the book vanishes, then comes back when the server rejects it. Explain why you cancel running queries first.

Stage 4: Rate a book (optimistic update on one field)

Build: five clickable stars on each book. Clicking a star sets the rating at once, with the same three callbacks as Stage 3, but changing one field of one book instead of removing a row.

Prove it: the stars change instantly. With the error switch on, they jump back to the old rating. Click three different stars quickly. The final rating on screen matches the server after the refetch.

Stage 5: Errors and feedback

Build:

- A "Simulate server errors" switch at the top of the page. When it is on, every change request sends the header `x-fail: true`. This is page state, not server data.
- A toast for each result: "Book added", "Could not remove [title]. It was restored." and so on. Use shadcn `sonner` or any small toast.
- An error message for each failed mutation, built from the thrown error.

Prove it: with the switch on, try all four actions. Each one shows a clear message, and the list never ends up in a wrong state.

Final questions he must be able to answer

1. What is the difference between a query and a mutation?
2. Why must the request function throw when `response.ok` is false?
3. What do `onMutate`, `onError` and `onSettled` each do?
4. What is the difference between `invalidateQueries` and `setQueryData`?
5. Why disable the Add button while the mutation is pending?